**Living source (Markdown):** D:\WingsFin-Test\README.md
**Repo path:** README.md · this PDF is a snapshot; the Markdown file remains the source of truth.

# WingsFin Real-Time Market Dashboard

Full-stack take-home assignment implementation for real-time index and stock charting. The system uses React, Vite, ECharts, Express, Socket.IO, Prisma, PostgreSQL, and Docker Compose.

## Features

- Market-open gate with configurable market hours.
- Default `DSEX` index chart and switchable `GP` stock chart.
- Backend-normalized one-minute history from open to current minute.
- Forward-filled missing minutes and latest-tick wins within the same minute.
- Irregular live simulator updates at unequal intervals up to 3 seconds.
- Dotted yesterday-close line, exact required point colors, tooltip, latest value, and heartbeat latest point.
- Manual simulation endpoints for index and stock payloads.

## Run With Docker

```
cp .env.example .env
docker compose up --build
```

If your local machine already uses PostgreSQL on `5432`, keep the internal Docker database connection unchanged and set the host port in root `.env`:

```
POSTGRES_PORT=55432
```

Open:

- Frontend: http://localhost:5173
- Backend health: http://localhost:4000/api/health
- Market status: http://localhost:4000/api/market/status
- API docs: http://localhost:4000/api/docs
- Audit trail: http://localhost:4000/api/audit/events

The backend container runs Prisma migrations and seeds non-uniform historical data before starting. API docs and audit reads are intentionally open during the development phase and should be protected once RBAC is implemented.

## Local Development

Backend:

```
cd backend
npm install
cp .env.example .env
npm run db:generate
npm run db:deploy
npm run seed
npm run dev
```

Make sure PostgreSQL is running before `db:deploy`. If you use the Compose Postgres service only, start it from the repo root with:

```
docker compose up -d postgres
```

Frontend:

```
cd frontend
npm install
cp .env.example .env
npm run dev
```

## Seed Data

With Docker:

```
docker compose exec backend npm run seed
```

Optional seed controls:

```
SEED_SESSION_DATE=2026-06-03
SEED_UNTIL_TIME=11:30
```

## Test Market Closed State

Set market hours outside the current time in `.env`, then restart:

```
MARKET_OPEN_TIME=10:00
MARKET_CLOSE_TIME=10:01
docker compose up --build
```

When closed, the frontend shows the closed-market message instead of charts.

## Manual Updates

Index:

```
curl -X POST http://localhost:4000/api/simulate/index ^
  -H "Content-Type: application/json" ^
  -d "{\"index_id\":\"DSEX\",\"capital_value\":5222.22,\"percentage_change_from_yesterday_close_value\":4.12}"
```

Stock:

```
curl -X POST http://localhost:4000/api/simulate/stock ^
  -H "Content-Type: application/json" ^
  -d "{\"trade_code\":\"GP\",\"close_price\":238.79,\"yesterday_close_price\":238.88}"
```

Updates are accepted only for timestamps inside the configured market session.

## Quality Checks

```
cd backend
npm run lint
npm run type-check
npm test
npm run build

cd ../frontend
npm run lint
npm test
npm run build
```

On Windows, stop any running backend dev server before `npm run build`. Prisma cannot replace its generated query-engine DLL while another Node process is using it.

## Design Decisions

- Raw ticks are stored instead of only aggregates so the chart can be rebuilt and audited.
- Historical data is normalized in the backend to keep API behavior deterministic.
- The frontend still merges live updates by minute so same-minute updates replace the current point and missing minutes are filled.
- Socket.IO avoids polling and emits only to subscribed symbol rooms.
- Market hours and timezone are environment-driven.
- PostgreSQL indexes on symbol and event time support scalable history queries.

## Trade-Offs

- The simulator runs inside the backend for assignment simplicity. In production, ingestion would likely be a worker or message consumer.
- Docker startup seeds data every time the backend starts so reviewers immediately see history. Production would separate seed/demo data from migrations.
- ECharts is bundled into the frontend, which makes the client bundle larger but gives strong time-series and effect-scatter support quickly.
- Prisma is pinned to v6 because v7 changed schema/client configuration significantly; v6 keeps the standard schema workflow clear for this assignment.

## Documentation

- docs/architecture.md — system architecture, data flows, database design, simulation strategy, scalability, trade-offs, and design decisions (with diagrams).
- docs/demo-script.md — step-by-step demo video script.

- docs/code_analysis/backend.md and docs/code_analysis/frontend.md — deeper, file-by-file onboarding guides for new engineers.
- API reference — interactive Swagger UI at http://localhost:4000/api/docs.

PDF copies of the architecture document, this README, and the demo script are provided alongside the submission email; the Markdown files above remain the living source of truth in the repo.

## Demo Video

Demo video link: *add submission video link here*.